

SISTEMAS DE COMPUTADORES – 2025/2026
Exame Época de Recurso

Versão A

Apenas autorizada a consulta da folha oficial
Duração: 30m

Nome: _____ Nº _____

Grupo I
(8/20 valores)

NOTAS:

- 1. Em todas as questões deverá assinalar apenas uma resposta.**
- 2. Se a resposta assinalada for incorreta sofrerá uma penalização de 1/3 da cotação da pergunta.**

1 – Após uma invocação bem-sucedida de `fork()`, qual das seguintes afirmações é correta?

- a) Pai e filho partilham o mesmo espaço de endereçamento virtual
- b) Pai e filho possuem tabelas de descritores de ficheiro distintas que referenciam as mesmas descrições de ficheiro abertas
- c) Todas as variáveis globais passam automaticamente a ser partilhadas
- d) O processo filho começa a executar a partir da função `main()`

2 – Considere um processo pai que cria um único filho através de `fork()`, mas não executa qualquer chamada a `wait()`. Qual é o estado do processo filho imediatamente após terminar a sua execução, assumindo que o processo pai continua em execução?

- a) Fica permanentemente em memória
- b) É automaticamente eliminado sem deixar qualquer informação
- c) Permanece temporariamente como processo *zombie* até que o pai termine ou execute `wait()`
- d) É adotado por um processo responsável por recolher processos órfãos após terminar

3 – Considere dois processos ligados por um *pipe*. O processo escritor fecha corretamente o descritor de escrita após enviar todos os dados. O processo leitor continua a executar chamadas sucessivas a `read()`. O que acontece quando todos os dados tiverem sido lidos?

- a) `read()` devolve -1, indicando que ocorreu um erro de leitura
- b) `read()` devolve 0, indicando que foi atingido o fim do fluxo de dados (EOF)
- c) O processo leitor bloqueia indefinidamente
- d) O *pipe* é automaticamente recriado

4 – Considere um semáforo contador inicializado com o valor 3. Três processos executaram com sucesso `sem_wait()`. Antes que qualquer um desses processos execute `sem_post()`, um quarto processo invoca `sem_wait()`. O que acontece?

- a) A invocação falha imediatamente
- b) O processo fica bloqueado até que outro processo execute `sem_post()`
- c) O valor do semáforo passa para -1 e a execução prossegue
- d) O semáforo é automaticamente reinicializado

5 - Considere uma implementação correta do problema produtor-consumidor baseada em semáforos. Qual das seguintes sequências de operações deve ser executada por um produtor antes de inserir um elemento no *buffer*?

- a) `sem_wait(empty); → sem_wait(mutex);`
- b) `sem_wait(mutex); → sem_wait(empty);`
- c) `sem_wait(full); → sem_wait(mutex);`
- d) `sem_wait(mutex); → sem_wait(full);`

6 - Numa implementação do problema escritor-leitor atribuindo prioridade aos escritores, qual é o principal objetivo do mecanismo de prioridade?

- a) Permitir que leitores e escritores acessem simultaneamente ao recurso
- b) Evitar que um escritor permaneça indefinidamente à espera enquanto chegam continuamente novos leitores
- c) Garantir que apenas um leitor pode aceder ao recurso de cada vez
- d) Eliminar completamente a necessidade de semáforos de exclusão mútua

7 - Qual das seguintes afirmações caracteriza corretamente um objeto do tipo `pthread_mutex_t`?

- a) Pode ser libertado por qualquer *thread*
- b) Deve ser libertado pela mesma *thread* que o adquiriu
- c) Permite que várias *threads* entrem simultaneamente na secção crítica
- d) É equivalente a uma variável de condição

8 - Qual das seguintes afirmações descreve corretamente uma diferença entre um objeto do tipo `pthread_cond_t` e um objeto do tipo `sem_t`?

- a) Um objeto do tipo `sem_t` mantém uma contagem, enquanto um objeto do tipo `pthread_cond_t` não mantém informação sobre sinais enviados quando não existem *threads* em espera
- b) Um objeto do tipo `pthread_cond_t` garante exclusão mútua, enquanto um objeto do tipo `sem_t` apenas sincroniza *threads*
- c) Um objeto do tipo `pthread_cond_t` pode ser utilizado sem um objeto do tipo `pthread_mutex_t`, ao contrário de um objeto do tipo `sem_t`
- d) Um objeto do tipo `pthread_cond_t` e um objeto do tipo `sem_t` são funcionalmente equivalentes e podem ser utilizados indistintamente

9 - Qual é o objetivo dos protocolos de coerência de cache em sistemas com múltiplos núcleos?

- a) Garantir que todas as caches têm a mesma capacidade
- b) Garantir que diferentes núcleos observam valores consistentes para uma mesma posição de memória
- c) Evitar a utilização de memória principal
- d) Eliminar completamente a falsa partilha (*false sharing*)

10 - Considere uma *thread* que modifica uma variável partilhada e, de seguida, executa `pthread_cond_signal(&cond)`. Porque é recomendável efetuar ambas as operações enquanto o `pthread_mutex_t` associado se encontra adquirido?

- a) Porque `pthread_cond_signal()` apenas pode ser chamado com o `mutex` adquirido
- b) Porque evita que outra *thread* observe um estado inconsistente da variável partilhada
- c) Porque impede a ocorrência de despertares espúrios (*spurious wakeups*)
- d) Porque garante que todas as *threads* em espera são acordadas

Versão A

Grupo II
(12/20 valores)

- 1. Responder cada questão numa folha separada, devidamente identificada.**
- 2. Indicar a versão do exame em cada uma das folhas.**
- 3. Não é necessário indicar o nome das bibliotecas usadas na resolução.**

- int parent_function(int parent_to_child[2][2],
child_to_parent[2][2], int *vec, int n);
- void child_function(int child_id, int parent_to_child[2][2],
child_to_parent[2][2]);

[7 valores] 2) A equipa na qual se encontra integrada tem como objetivo desenvolver um programa em C que simule uma fila de impressão partilhada entre vários processos. Existem vários processos produtores, representando utilizadores que submetem trabalhos de impressão, e vários processos consumidores, representando impressoras. Cada trabalho de impressão é representado por um número inteiro correspondente ao seu identificador.

Os produtores devem:

- inserir trabalhos numa fila circular com capacidade limitada;
- caso a fila esteja cheia, o **produtor deve bloquear** até existir espaço disponível.

As impressoras devem:

- remover trabalhos da fila;
- caso a fila esteja vazia, a **impressora deve bloquear** até existir pelo menos um trabalho disponível.

A sincronização deve ser realizada **exclusivamente utilizando memória partilhada e semáforos POSIX, não sendo permitida espera ativa** (*busy waiting*). Assuma que a estrutura de dados da fila já foi implementada e inicializada corretamente:

```
#define BUFFER_SIZE 10

typedef struct {
    int jobs[BUFFER_SIZE];
    int head;
    int tail;

    sem_t mutex;
    sem_t empty;
    sem_t full;
} shared_memory_t;
```

Assuma ainda que:

- a memória partilhada já foi criada e mapeada;
- o apontador `shared_memory_t *shared_memory` já aponta para essa zona de memória;
- todos os semáforos já foram corretamente inicializados com os seguintes valores:
 - `mutex` com o valor 1;
 - `empty` com o valor `BUFFER_SIZE`;
 - `full` com o valor 0.
- a fila se encontra inicialmente vazia;
- existe apenas uma fila partilhada por todos os processos.

Foram-lhe atribuídas as tarefas de desenvolvimento das funções responsáveis pela inserção e remoção de trabalhos, pelo que deverá implementar **apenas** estas:

- `void submit_job(shared_memory_t *shared_memory, int job_id);`
- `int get_job(shared_memory_t *shared_memory);`